

DC HOBBIES

CULTURA GEEK ONLINE

SPRINT 1 · ENTREGABLE

Documento del Sprint 1

Administración de productos e inicio de la interfaz de cliente

INTEGRANTE	CARNÉ	ROL SCRUM
Joaquín Rodríguez Contreras	C4J075	Product Owner
Juan Loaiza	B74200	Scrum Master
Agustín Soto	C4K199	Developer
Jeferson Marín	C24549	Developer

Equipo: Twenty One CoPilots

Sprint: 22 de setiembre – 7 de octubre de 2026

Documento base: SRS PI-REQ-001 v3.1 (Sprint 0)

Cliente: Daniel Cabezas

Fecha del documento: 4 de octubre de 2026 · v1.1

Repositorio:
github.com/juanselospi/21CoPilots_PIBasesIngesoft

Índice

1. Introducción	2
2. Resumen de lo implementado	3
2.1 Vista general del sistema	3
2.2 Panel administrativo de productos	3
2.3 Motor de precios	4
2.4 Interfaz del cliente	4
2.5 Inicio de sesión y permisos	5
2.6 Base de datos	5
2.7 Reglas de negocio y validaciones aplicadas	5
2.8 Lo que no quedó completo y por qué	6
2.9 Guía rápida para revisar el sistema	7
3. Cambios respecto al Sprint 0 y su justificación	8
3.1 Cómo llegamos a estos cambios	8
3.2 Cambios al modelo de datos	8
3.3 Esquema relacional implementado	9
3.4 Cambios en la arquitectura	11
3.5 Cambios de alcance y de requerimientos	11
4. Evidencia de las pruebas	13
4.1 Entorno y datos de prueba	13
4.2 Pruebas automatizadas	13
4.3 Casos de prueba funcionales	14
4.4 Capturas de pantalla	16
4.5 Pruebas de la API	22
5. Sprint Backlog del Sprint 1	24
5.1 Tareas del sprint	24
5.2 Tareas que quedaron obsoletas	26
6. Product Backlog actualizado	28
7. Historias de usuario y criterios de aceptación abordados	30
HU-01 — Importar el catálogo desde Excel	30
HU-02 — Iniciar sesión con permisos por rol	31
HU-03 — Registrar y clasificar productos	31
HU-04 — Calcular el precio de venta automáticamente	32
HU-05 — Consultar el catálogo con disponibilidad real	32
Historias agregadas en el sprint (aprobadas el 24/09)	33
8. Retrospectiva del equipo	34
9. Pendientes para el Sprint 2	35

1. Introducción

Este documento resume lo que hicimos en el Sprint 1 del proyecto **DC Hobbies Cultura Geek Online**. El Sprint 1 fue el primero de implementación: en el Sprint 0 levantamos los requerimientos (SRS), el Product Backlog, el EER con su mapeo y elegimos el stack, y en este sprint construimos sobre esa base la primera versión que funciona de verdad.

El objetivo que pusimos en Jira al planificar el sprint fue: «*Administrar el catálogo de productos de punta a punta (ingresar, modificar, excluir, consultar) y mostrarlo al cliente con su disponibilidad real*». Para lograrlo tomamos del backlog las cinco historias que el Sprint 0 había propuesto para este sprint: HU-01 (importar el Excel), HU-02 (inicio de sesión), HU-03 (registrar productos), HU-04 (cálculo del precio) y HU-05 (catálogo con disponibilidad real).

A mitad del sprint, el 24 de setiembre, tuvimos una entrevista con el cliente donde le presentamos los wireframes y 16 funcionalidades que no estaban en el SRS. El cliente las aprobó todas, y dos de ellas (la página de inicio y los filtros del panel) las metimos en este sprint porque estaban ligadas directamente al catálogo. Las demás quedaron en el Product Backlog (sección 6).

Resultado del sprint en pocas palabras

- **Ingresar y consultar productos:** listos. El administrador puede agregar productos con un formulario o cargar su hoja de Excel, y puede buscar, filtrar y ordenar todo su catálogo desde el panel.
- **Modificar:** listo para los datos que forman el precio (costo, importación y margen), las existencias y el contrapedido, con el precio y la disponibilidad recalculados en el momento. Cambiar el nombre, la descripción, la categoría, el proveedor o la imagen todavía no se puede desde el panel.
- **Excluir:** no se implementó. Depende de una decisión sobre el modelo de datos que explicamos en la sección 2.8.
- **Interfaz del cliente:** página de inicio, catálogo completo, catálogo por categoría y ficha de producto, conectadas a la base de datos y con la disponibilidad real de cada producto.
- **Pruebas:** 167 pruebas automatizadas del servidor, 86 de la base de datos y 20 casos funcionales con capturas. Todas pasan.
- **Sprint Backlog:** 53 de 55 tareas terminadas; 24 de los 34 puntos planeados quedaron completos (HU-01, HU-04 y HU-05), y HU-02 y HU-03 quedaron parciales.

El documento sigue el orden que piden las instrucciones del Sprint 1: primero lo implementado (sección 2), después los cambios respecto al Sprint 0 (sección 3), la evidencia de pruebas (sección 4), los artefactos de Scrum (secciones 5, 6 y 7) y al final la retrospectiva y los pendientes (secciones 8 y 9).

2. Resumen de lo implementado

2.1 Vista general del sistema

Mantuvimos la arquitectura que definimos: una aplicación web en React que habla con un servidor Express por una API REST, y el servidor guarda todo en PostgreSQL. El servidor está organizado por módulos (catálogo, admin, inventario, pedidos, etc.) y cada módulo tiene sus capas de rutas, controlador, servicio y repositorio. En la base de datos cada módulo tiene su propio esquema.



2.2 Panel administrativo de productos

El panel está en `/admin/productos` y solo entra el administrador. Desde ahí se hacen las operaciones sobre la entidad Producto:

Operación	Qué puede hacer el administrador	Endpoint
Consultar	Ver todos sus productos (incluidos los que la tienda oculta) en cuadrícula o en tabla. Puede buscar por nombre o por código, filtrar por categoría, disponibilidad, rango de precio final, existencias bajas y margen negativo, ordenar por nombre, precio, existencias o código, y paginar de 24, 48 o 96. La tabla muestra etiquetas: <i>por contrapedido, existencias bajas, oculto en la tienda y margen negativo</i> .	GET <code>/api/catalogo/admin/productos</code>
Ingresar (uno por uno)	Formulario «Agregar producto» con código, nombre, categoría, proveedor, imagen, descripción, costo, importación, margen, existencias (opcional; si no se indican, entra con 0) y si admite contrapedido. El precio final no se escribe: lo calcula el servidor al guardar y el formulario lo muestra en el mensaje de éxito.	POST <code>/api/catalogo/productos</code>
Ingresar (en lote)	«Cargar Excel» acepta la misma hoja que el negocio ya usa, con sus columnas y en dólares (también hay una plantilla descargable). Se validan todas las filas antes de guardar, las válidas se guardan en una sola transacción, los productos que ya existen se actualizan por su código y las filas con problemas salen en un reporte descargable con el motivo.	POST <code>/api/admin/importaciones</code> GET <code>/api/admin/importaciones/plantilla</code>

Modificar	Modal «Editar producto»: cambia el costo del ítem, el porcentaje de importación, el margen, las existencias o si admite contrapedido, y muestra el costo total, el precio sin impuesto y el precio final ya recalculados por el servidor.	PATCH <code>/api/catalogo/productos/:sku</code>
Excluir	No implementado Ver sección 2.8.	—

2.3 Motor de precios

El precio de venta nunca se escribe a mano ni se guarda en la base de datos: lo calcula un motor de precios en el servidor a partir del costo, la importación y el margen de cada producto. Lo armamos como una cadena de pasos (importación → margen → impuesto), para que si la fórmula cambia solo haya que tocar un paso. La fórmula la documentamos con el PO en `formula-precio.md` (SCRUM-22) y la validamos contra el Excel real del cliente, lo que cerró la inconsistencia INC-05 que traíamos del Sprint 0.

Paso	Cálculo	Ejemplo (\$49,99, 20 % y 20 %)
1. Costo total	costo del ítem + % de importación	\$59,99
2. Precio sin impuesto	costo total + % de margen (puede ser negativo, pero mayor que -100 %)	\$71,99
3. Precio final	precio sin impuesto + 13 % de impuesto de venta	\$81,34

Un detalle que costó encontrar: el Excel del cliente calcula con el valor exacto y redondea solo lo que muestra. Si se redondea en cada paso el precio da \$81,35 y ya no coincide con la hoja. Lo arreglamos en SCRUM-25 y ahora el sistema da exactamente lo mismo que el Excel (caso PF-04).

2.4 Interfaz del cliente

La tienda es pública: el visitante la recorre sin iniciar sesión. Todo lo que muestra sale de la misma base de datos que administra el panel, así que un cambio del administrador se ve de inmediato en la tienda.

- **Página de inicio** (`/`): banner del negocio, accesos a las categorías, un bloque «Recién Llegados» y una sección por categoría con sus productos (aprobado en la entrevista, puntos 8 y 16).
- **Catálogo completo y por categoría** (`/catalogo` y `/catalogo/:categoria`): tarjetas con imagen, nombre, precio con impuesto incluido y el rótulo de disponibilidad.
- **Ficha de producto** (`/producto/:sku`): imagen, nombre, precio final, disponibilidad y descripción.
- **Disponibilidad real (RN-03)**: un producto con existencias sale como *En existencia*; sin existencias pero con contrapedido sale como *Por contrapedido*; y sin existencias ni contrapedido no aparece en ningún lado de la tienda (ni en el catálogo, ni en su categoría, ni por URL directa). Las categorías también cuentan solo los productos visibles, así que una categoría con todo oculto no sale en el menú.
- **Identidad visual**: aplicamos la propuesta que eligió el cliente (paleta «Slime» y letra de rótulo). El logo definitivo todavía no existe y por ahora se muestra el nombre.

Los botones «Agregar al carrito», el contador del carrito y el buscador de la barra superior aparecen porque son parte del wireframe, pero no hacen nada todavía: el carrito es HU-10 y la búsqueda del cliente es HU-06, que son de sprints posteriores. No quisimos adelantar trabajo fuera del alcance.

2.5 Inicio de sesión y permisos

Para proteger el panel implementamos la parte principal de HU-02. El usuario inicia sesión en `/acceso`; el servidor revisa la contraseña (guardada con hash bcrypt, nunca en texto plano) y abre una sesión con una cookie `HttpOnly` que dura 25 minutos. En la base de datos solo se guarda el hash del token de la sesión.

Las rutas del panel pasan por dos middlewares: `exigirSesion` (sin sesión responde 401) y `exigirRol(administrador)` (con otro rol responde 403). En el cliente, `AdminRoute` manda al visitante a la pantalla de acceso y al cliente de vuelta a la tienda, pero esto es solo para la experiencia: la protección real es la del servidor (casos PF-06 a PF-08 y API-01 a API-03). Las reglas están resumidas en `matriz-permisos.md` (SCRUM-6).

2.6 Base de datos

El esquema relacional del Sprint 0 quedó materializado en 9 migraciones SQL numeradas (`database/migraciones/`), una por módulo, con 14 tablas y 5 vistas de reportes. El DDL completo se genera con `npm run db:dump` en `database/schema.sql`, que es el script que entregamos. Además hay semillas con datos de prueba (usuarios, productos que cubren los casos del SRS, mercancía y pedidos de ejemplo) y 86 pruebas SQL que verifican que cada restricción rechaza lo que tiene que rechazar.

Para que cualquiera del equipo (o alguien externo) levante todo igual, PostgreSQL 17 corre en Docker y todo se maneja con comandos: `db:up`, `db:reset`, `db:test`, `db:dump` y `db:cliente` (este último reconstruye la base y carga el Excel del cliente por el servidor).

2.7 Reglas de negocio y validaciones aplicadas

Seguimos la regla que definimos en la arquitectura: **la base de datos garantiza integridad y el servidor decide las reglas de negocio**. Por eso varias reglas están en las dos capas: el servidor da un mensaje claro y la base de datos es la segunda línea de defensa.

Regla	Dónde se aplica	Evidencia
Código, nombre, categoría y costo obligatorios	Servidor (<code>validarProductoNuevo</code> , junta todos los errores en una respuesta) y <code>CHECK</code> en la tabla	API-04
Código único (RF-01), sin importar mayúsculas ni espacios	El servidor normaliza a mayúsculas y responde 409; la BD tiene la llave primaria y un <code>CHECK</code> de normalización	PF-16, API-05
Costo e importación no negativos	Servidor y <code>ck_producto_item</code> , <code>ck_producto_importacion</code>	API-04
Margen negativo permitido pero mayor que -100 % (RN-02)	Formulario, servidor y <code>ck_producto_margen</code>	PF-17, API-06
Existencias enteras y no negativas	Formulario, servidor (al agregar y al editar) y <code>ck_producto_stock</code> en la BD	Pruebas <code>validar-cambios-de-producto</code> y <code>01_catalogo.sql</code>
El precio no se puede escribir a mano (RN-01)	El PATCH rechaza cualquier campo que no sea costo, importación, margen, existencias o contrapedido	API-07
Producto sin existencias ni contrapedido no se muestra (RN-03)	Consultas públicas del catálogo y de las categorías	PF-02, PF-05, API-09

Alerta con 2 unidades o menos (RN-04)	Etiqueta «Existencias bajas» y filtro en el panel	PF-10
Solo el administrador administra productos (RF-50)	Middlewares del servidor y rutas protegidas del cliente	PF-06, PF-08, API-01 a 03
Importación: las filas inválidas no se guardan y se reportan (RF-58); reimportar no duplica (RF-59)	Validación de todo el archivo antes de guardar, transacción y <code>ON CONFLICT (sku)</code>	PF-13, PF-14
El registro de mercancía no se edita ni se borra (RF-19)	Trigger de solo inserción en <code>producto_administra</code>	Prueba <code>02_inventario.sql</code>

2.8 Lo que no quedó completo y por qué

Excluir (dar de baja) un producto — RF-18

No lo implementamos. En el Sprint 0 la idea era una baja lógica con una columna `estado` (*activo* o *descontinuado*). Cuando el 30 de setiembre volvimos al EER del equipo (sección 3.1), esa columna no estaba en el diagrama, y nos quedaron tres opciones:

- Volver a agregar `estado` al EER con una migración nueva. Es la única que distingue un producto descontinuado de uno agotado.
- No tocar el modelo y dejarlo con `stock = 0` y sin contrapedido para que RN-03 lo oculte. El problema es que se confunde con un producto agotado y vuelve a aparecer si entra mercancía.
- Borrarlo físicamente. No sirve: las llaves foráneas de `agrega` y `producto_administra` lo impiden y se perdería el historial de ventas.

Como la decisión cambia el modelo o el significado de los datos, preferimos no improvisarla a última hora. Sabemos que es una de las cuatro operaciones que pide el enunciado, así que es el **primer punto del Sprint 2**: nuestra propuesta es la opción (a), con la migración `010`, la ruta para descontinuar y el filtro `descontinuado` en el listado del panel, que ya está previsto en el servidor.

- **Modificar los datos generales del producto.** El modal edita los campos del precio, que eran los de HU-04 y RF-01, además de las existencias y el contrapedido. Cambiar nombre, descripción, categoría, proveedor o imagen se hace por ahora volviendo a importar el Excel (que actualiza por código), y la edición desde el panel pasa al Sprint 2.
- **Existencias al importar.** La hoja del cliente no trae la cantidad, así que un producto importado entra con 0 existencias y solo se ve en la tienda si admite contrapedido. En el panel se pueden indicar las existencias al agregar un producto y corregirlas al editarlo, pero esa corrección no queda en el registro de mercancía (`producto_administra`): las entradas con su historial son del ingreso de mercancía (HU-07, Sprint 2). Para las pruebas y la demostración usamos `npm run db:cliente`, que deja una unidad a cada producto importado.
- **Paginación del catálogo público.** «Ver todo» muestra hasta 48 productos, que es el máximo de una página. Con el catálogo real (~195 SKU) hace falta que la API devuelva el total y decidir entre números de página o «Cargar más».
- **«Recién llegados»** muestra los primeros cinco productos del catálogo, no los últimos que entraron, porque eso depende de la fecha de ingreso de mercancía (HU-07).

2.9 Guía rápida para revisar el sistema

Las instrucciones plantean seis preguntas que cualquier persona debería poder comprobar. Esta tabla dice dónde está la respuesta de cada una:

Pregunta	Respuesta corta	Dónde verlo
¿Cómo ingresa el personal administrativo un nuevo producto?	Panel → «Agregar producto» (uno por uno) o «Cargar Excel» (en lote).	§ 2.2, PF-13 a PF-15
¿Cómo modifica la información de un producto?	Panel → «Editar» en la tarjeta o en la tabla → cambia costo, importación, margen, existencias o contrapedido. El resto de datos, por reimportación (§ 2.8).	§ 2.2, PF-14, PF-18
¿Cómo excluye un producto sin perder el historial?	Todavía no se puede; la propuesta es una baja lógica con columna estado .	§ 2.8
¿Cómo consulta y filtra los productos?	Panel → buscador por nombre o código, botón «Filtros», orden y vista de tabla.	PF-09 a PF-12
¿Cómo ve el cliente el catálogo?	Tienda pública: inicio, «Ver todo», categorías del menú y ficha de cada producto.	§ 2.4, PF-01 a PF-05
¿Qué cambió del diseño del Sprint 0 y por qué?	Ajustes al modelo de datos y al alcance, todos con su justificación.	§ 3

Usuarios de prueba (semillas): administrador `admin@dchobbies.test` / `Admin123!` y cliente `cliente@correo.test` / `Cliente123!`. Los pasos para levantar el sistema están en el `README.md` del repositorio y en `database/guia.md`.

3. Cambios respecto al Sprint 0 y su justificación

3.1 Cómo llegamos a estos cambios

Al empezar el sprint (26 y 27 de setiembre) armamos una propuesta del modelo de datos bastante distinta al EER del Sprint 0: llaves numéricas en vez de correo y SKU, tablas de categoría y subcategoría, un libro de movimientos de inventario, baja lógica de productos, niveles de fidelidad y parámetros del negocio en tablas, entre otras cosas. Llegamos a montarla en la base de datos y empezamos a programar el servidor sobre ella.

Al revisarla con calma vimos que era casi un rediseño completo, y las instrucciones del Sprint 1 piden que los ajustes al modelo sean **mínimos y justificados**. Además el resto del equipo seguía trabajando con el EER del Sprint 0. Por eso el 30 de setiembre decidimos volver al EER del equipo y corregirlo (lo llamamos «EER corregido»): la base de datos ahora sigue ese diagrama al pie de la letra y adaptamos al nuevo modelo el inicio de sesión, el catálogo y la importación. Los cambios que quedaron son los de la sección 3.2.

Esta vuelta atrás tuvo un costo: varias tareas de Jira quedaron obsoletas (sección 5.2) y los documentos `modelo-datos.md` y `arquitectura.md` §9 quedaron describiendo la propuesta que descartamos hasta el 4 de octubre, cuando los pasamos al EER corregido. Lo comentamos también en la retrospectiva.

3.2 Cambios al modelo de datos

ID	Qué cambió	Por qué	Requerimientos
C-01	Producto: <i>Precio Venta</i> y <i>Precio Impuesto</i> ya no se guardan; los calcula el motor de precios.	Son atributos derivados. Si se guardan, quedan desactualizados cada vez que cambia el costo o el margen, y la fórmula todavía estaba en discusión (INC-05). Con el motor solo hay un lugar que calcula el precio.	RN-01, RF-03 a RF-05, INC-05
C-02	Producto: <i>Costo Total</i> pasa a ser una columna generada ($\text{item} \times (1 + \text{importación} / 100)$) y se agrega <i>tasa_impuesto</i> (13 % por defecto).	En el EER, Costo Total es un atributo compuesto de Item e Importación; como columna generada nunca puede contradecir a sus partes. La tasa se guarda para que cada línea del carrito fije la que se le aplicó.	RN-01, RES-06
C-03	Montos en dólares en vez de colones.	El PO confirmó el 1 de octubre que la tienda maneja dólares, y la hoja del cliente trae la columna <code>costo_usd</code> . Falta confirmar si la factura y los pagos también van en dólares (RNF-17 decía colones).	RNF-17
C-04	Categoría como texto, sin subcategoría.	El SRS (RF-02) y HU-03 hablaban de categoría y subcategoría, pero el EER del Sprint 0 ya tenía Categoría como atributo simple y la hoja real del cliente solo trae la «familia». Lo confirmamos y se descartó la subcategoría.	RF-02, RF-10
C-05	Carrito: llave (<code>correo_cliente</code> , <code>num_carrito</code>) como entidad débil de Cliente; se agregan <i>Estado carrito</i> (<code>activo / convertido</code>) y <i>Fecha cierre</i> ; solo un carrito activo por cliente.	El EER del Sprint 0 ya dibujaba Carrito como entidad débil («Tiene»), pero el mapeo usaba un <code>CarritoId</code> propio. También hace falta saber qué carrito ya se convirtió en pedido.	RN-14, RF-24

C-06	Pedido: pasa a ser entidad débil de Carrito (relación «Convierte», 1 a 0..1) y hereda su llave. Se elimina <i>NumeroPedido</i> y la tabla <i>PEDIDO_PRODUCTO</i> («Incluye»): las líneas del pedido son las de <i>Agrega</i> .	Todo pedido en línea nace de un carrito, y tener las mismas líneas en dos tablas era duplicar datos. Las ventas de otros canales (RF-17) también pasan por un carrito; el detalle se define con HU-09 en el Sprint 2.	RF-25, RF-30, RF-17
C-07	Agrega: guarda <i>Precio unitario</i> (sin impuesto) y <i>Tasa impuesto aplicada</i> .	El precio que se cobra queda fijo en la línea: si después cambia el margen o el costo, los pedidos ya hechos no cambian.	RF-25, RF-51
C-08	Entidad nueva Oferta (relación «Aplicar» con Pedido): vigencia, porcentaje, monto mínimo y nivel de fidelidad mínimo.	En la entrevista del 24/09 (punto 10) el cliente pidió poder ajustar los descuentos sin cambiar el sistema. Reemplaza el monto mínimo fijo de RN-09.	RN-08, RN-09, aprobado #10
C-09	Entidad nueva Factura («Respalda»: un pago tiene 0 o 1 factura). Subtotal, impuestos, descuento y total son derivados.	El SRS pide factura por cada pago aprobado y el EER no la tenía.	RF-41, RN-18
C-10	Entidad nueva Sesión , débil de Usuario: hash del token, fecha de creación y de vencimiento.	El inicio de sesión usa una cookie con un token aleatorio. Guardar solo su hash permite validar la sesión en cada petición sin exponer tokens si alguien lee la tabla.	RF-53, RNF-08
C-11	Producto_Administra (relación «Administrar»): la cantidad va con signo (distinta de 0), la llave incluye la fecha y es de solo inserción.	Así sirve para entradas y salidas de mercancía, y es un historial que no se puede editar (RF-19).	RF-13, RF-15, RF-19
C-12	Entidad débil Historial del estado (relación «Actualiza» con Pedido): llave parcial <code>estado</code> y atributo <code>fecha</code> ; <code>CHECK</code> con los estados de RF-26 más <code>cancelado</code> , solo inserción. El estado del pedido se lee de la vista <code>v_estado_pedido</code> .	El estado ya era un atributo derivado en el EER del Sprint 0; ahora queda acotado a los estados del SRS, se sabe cuándo ocurrió cada cambio y su historial no se puede alterar.	RF-26, RF-28
C-13	Implementación física (no cambia el EER): un esquema de PostgreSQL por módulo, restricciones con nombre, correo y SKU normalizados, índice que permite un solo administrador, triggers de solo inserción y vistas para los reportes.	Alinea la base de datos con los módulos del servidor, hace verificables RF-19 y RF-49, y permite que el servidor traduzca cada error a un mensaje de negocio.	RF-19, RF-49, RNF-19
C-14	Usuario y Sesión van en un esquema propio, <code>usuarios</code> , en vez de <code>admin</code> (migración <code>002_usuarios.sql</code>).	Las cuentas y sus sesiones son de todos los usuarios, clientes incluidos. En <code>admin</code> hacían pensar que todo usuario era administrador.	RF-50, RF-53

3.3 Esquema relacional implementado

Este es el esquema que quedó en las migraciones (y en `schema.sql`). Las tablas nuevas respecto al Sprint 0 tienen la marca *nueva* y las columnas que cambiaron están resaltadas. El diagrama EER actualizado es el archivo `EER_21_CoPilots.drawio.xml` que se entrega aparte; como comparación, la figura 1 es el EER del Sprint 0.

sku llave primaria → tabla llave foránea columna nueva o cambiada

usuario usuarios <u>correo</u> contrasena hash bcrypt nombre	administrador admin <u>correo_usuario</u> → usuario índice: un solo administrador	sesion nueva <u>correo_usuario</u> → usuario <u>token_hash</u> fecha_creacion fecha_vencimiento
cliente clientes <u>correo_usuario</u> → usuario cedula única direccion num_compras	cliente_telefono clientes <u>correo_usuario</u> → cliente <u>telefono</u>	producto catalogo <u>sku</u> normalizado nombre · categoria · proveedor descripcion · imagen item · importacion · margen_ganancia costo_total generada tasa_impuesto stock · contrapedido
producto_administra inventario <u>sku</u> → producto <u>correo_administrador</u> → administrador <u>fecha</u> cantidad con signo	carrito pedidos <u>correo_cliente</u> → cliente <u>num_carrito</u> estado_carrito fecha_creacion · fecha_actualizacion fecha_cierre	agrega pedidos <u>correo_cliente, num_carrito</u> → carrito <u>sku</u> → producto cantidad_solicitada precio_unitario tasa_impuesto_aplicada
pedido pedidos <u>correo_cliente, num_carrito</u> → carrito fecha_pedido_realizado · fecha_entrega modalidad_entrega · direccion costo_entrega codigo_oferta → oferta	oferta nueva <u>codigo_oferta</u> nombre · descripcion fecha_inicio · fecha_fin porcentaje_descuento monto_minimo · nivel_fidelidad_minimo	historial_estado pedidos <u>correo_cliente, num_carrito</u> → pedido <u>numero_cambio</u> estado CHECK RF-26 fecha
pago pagos <u>num_referencia</u> fecha · monto metodo_pago · estado_pago correo_cliente, num_carrito → pedido	factura nueva <u>num_factura</u> fecha_emision num_referencia → pago (única)	vistas reportes v_estado_pedido v_venta_por_producto v_existencias v_pedidos_por_cliente v_registro_mercancia

Dos puntos de las migraciones que todavía no siguen el EER. El EER marca la especialización de Usuario en Administrador y Cliente como traslapada, pero `004_clientes.sql` tiene un trigger que impide que una cuenta sea las dos cosas a la vez. Y en el EER la llave parcial de Historial del estado es `estado`, mientras que la migración usa `numero_cambio` (la página «Mapeo» del diagrama también lo tiene así). Los dos se corrigen con una migración nueva en el Sprint 2 (sección 9); no afectan lo implementado en este sprint, porque pedidos todavía no se usa y en los datos de prueba nadie es administrador y cliente.

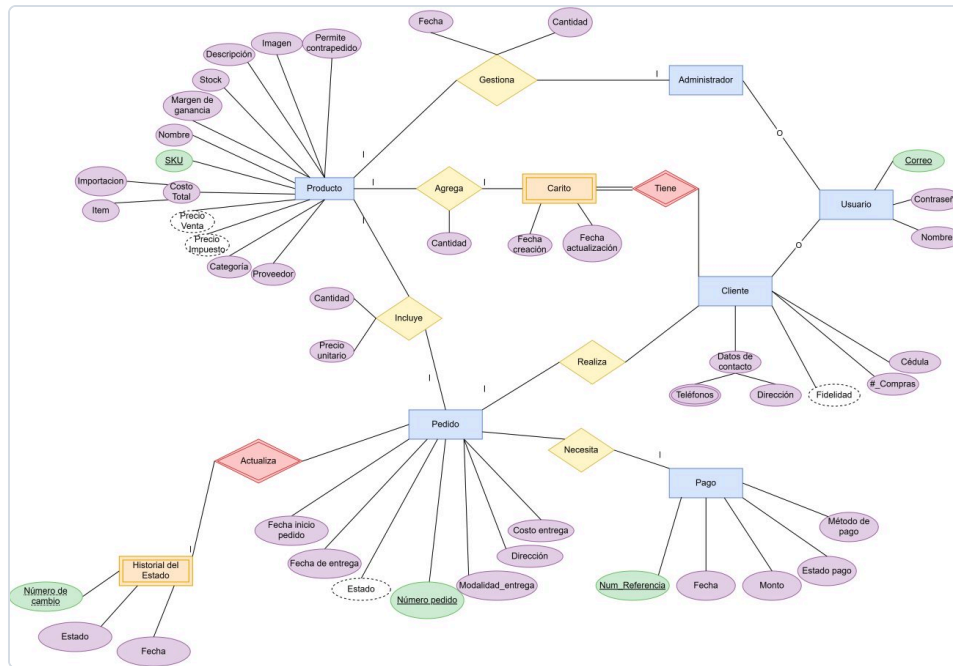


Figura 1. EER entregado en el Sprint 0, punto de partida de los cambios C-01 a C-12.

3.4 Cambios en la arquitectura

La arquitectura no cambió en lo grande: seguimos con React + Vite, Node.js + Express y PostgreSQL, con el servidor en capas por módulo y el módulo de catálogo como plantilla para los demás. Los ajustes fueron pequeños:

- **Bitácora de auditoría (RF-52):** el EER corregido no tiene una entidad Bitácora, así que quitamos el suscriptor que la escribía. Los eventos de dominio se siguen publicando, y falta decidir si la bitácora va en una tabla nueva o en un archivo de log.
- **Escala de fidelidad y datos fiscales:** la propuesta descartada los tenía en tablas; con el EER corregido la propuesta es pasarlos a la configuración del servidor. Se usan en descuentos y facturación, que no son de este sprint.
- **Módulos que todavía son esqueletos** (inventario, clientes, pedidos, pagos, facturación y reportes): sus encabezados todavía hablan del modelo anterior y se adaptan al EER corregido antes de implementarlos.
- **Cliente web:** el componente de importación todavía llama a la API directamente, lo que incumple la regla de `arquitectura.md` § 11.1 (ningún componente importa de `api /`). Lo dejamos anotado para corregirlo.

3.5 Cambios de alcance y de requerimientos

Cambio	Justificación	Efecto
Página de inicio con vitrina y secciones por categoría	Aprobada por el cliente el 24/09 (puntos 8 y 16). No estaba en ningún RF.	Agregada a S1
Filtros y orden del panel (existencias bajas, margen negativo, existencias)	Aprobado el 24/09 (punto 9): los filtros del SRS (RF-11) eran para el catálogo público.	Agregada a S1
Recuperación de contraseña (RF-54)	Es <i>Should</i> y el sprint se enfocó en productos.	Pasa a S2

Cuentas para empleados	Aprobada el 24/09 (punto 6), pero los empleados usan sobre todo ventas y pedidos, que vienen después.	Pasa a S2
Movimientos de inventario y alerta de existencias bajas en el panel	No cabían en el sprint; la etiqueta de existencias bajas sí quedó en el listado.	Pasa a S2
Cierre de sesión y expiración	No corresponde a ningún RF del SRS; se reevalúa junto con RF-54 y la política de contraseñas que dio el cliente.	Por reevaluar
Subcategorías e importación de la cantidad	La hoja real del cliente no las trae (C-04 y § 2.8).	Criterio ajustado

4. Evidencia de las pruebas

4.1 Entorno y datos de prueba

Todas las pruebas se corrieron el 4 de octubre de 2026 sobre la rama `develop` (commit `f44f20d`, el último merge a esa fecha), con Node.js 22 y PostgreSQL 17 en Docker. Para las pruebas funcionales reconstruimos una base de datos limpia con `npm run db:cliente` y le cargamos un Excel de prueba con 13 productos en el mismo formato de la hoja del cliente (montos en dólares, cuatro familias). Después le ajustamos las existencias registrando ingresos de mercancía, para que estuvieran todos los casos del SRS:

Producto	Caso que cubre	Regla
MTG-CMD-001	Sin existencias, admite contrapedido: se ve como «Por contrapedido»	RN-03
FUN-POP-LTD	Sin existencias ni contrapedido: no se ve en la tienda	RN-03
TCG-SLV-DAN	Margen de -10 % (liquidación), se vende bajo el costo	RN-02
FUN-POP-GOKU	Dos unidades: dispara la etiqueta de existencias bajas	RN-04
PS5-GOW-RAG	Producto del ejemplo real del Excel (\$49,99, 20 %, 20 % → \$81,34)	RN-01

Para que las pruebas se puedan repetir, el recorrido por las pantallas lo hicimos con un script de navegador (Playwright) que sigue los mismos pasos que haría una persona y toma la captura de cada resultado. Las imágenes de los productos son de relleno (placeholder.co).

4.2 Pruebas automatizadas

Además de las pruebas manuales, el proyecto tiene pruebas automatizadas que cualquiera puede correr con un solo comando. Los nombres de las pruebas citan el requerimiento que verifican (por ejemplo, «*RF-59: reimportar la misma hoja actualiza y no crea productos*»).

Tabla 1. Pruebas del servidor (`npm test` , `node:test`)

Archivo	Pruebas
catalogo.service	31
importacion-de-excel	29
validar-producto-nuevo	22
validar-cambios-de-producto	17
catalogo.routes	15
validar-consulta-administrativa	12
importacion.routes	9
motor-de-precios	8
admin.service	6
sesion.routes · contraseñas	4 · 4
maquina-de-estados · bus-de-eventos	4 · 4
reporte-de-rechazos	2
Total	167 / 167 ✓

Tabla 2. Pruebas de la base de datos (`npm run db:test`)

Archivo	Pruebas
01_catalogo	13
02_inventario	9
03_usuarios_clientes	16
04_pedidos	26
05_pagos_facturacion	10
06_invariantes	4
07_sesion	8
Total	86 / 86 ✓

Cada archivo corre dentro de una transacción que se revierte al final, así que no deja datos. Verifican, por ejemplo, que un SKU repetido o un stock negativo se rechacen y que el historial de mercancía no se pueda editar ni borrar.

```
$ cd apps/server && npm test
✓ RF-01: registrar PKM-001 dos veces da error la segunda vez
✓ RF-03, RF-04, RF-05: con costo 1000, importación 20 y margen 25 el precio final es 1695
✓ RN-02: el margen tiene que ser mayor que -100 %
✓ RF-58: importa las válidas, rechaza las inválidas e informa el total
✓ RF-59: reimportar la misma hoja actualiza y no crea productos
✓ RN-03: las categorías solo cuentan los productos visibles
✓ RNF-08: la misma contraseña produce hashes distintos (salt)
... 160 más
i tests 167 i pass 167 i fail 0

$ cd database && npm run db:test
→ 02_inventario.sql ... → 07_sesion.sql
✓ El historial de estados no se puede editar
✓ Un hash de token repetido se rechaza, aunque sea de otro usuario
✓ 86 prueba(s) aprobada(s).
```

4.3 Casos de prueba funcionales

ID	Qué se probó	Resultado esperado	Resultado obtenido	Estado
Interfaz del cliente (HU-05)				
PF-01	Página de inicio como visitante	Banner, categorías del catálogo y productos agrupados	Se ven las 4 familias y sus productos	Pasa
PF-02	«Ver todo» el catálogo	Solo productos visibles; el oculto no aparece; el de contrapedido rotulado	12 de 13 productos; FUN-POP-LTD no aparece; MTG-CMD-001 dice «Por contrapedido»	Pasa
PF-03	Catálogo de la categoría Legos	Solo productos de Legos	Los 3 sets de LEGO	Pasa

PF-04	Ficha de un producto con existencias	Imagen, nombre, descripción, precio final igual al Excel y «En existencia»	\$81,34, igual que la hoja del cliente	Pasa
PF-04b	Ficha de un producto por contrapedido	Rótulo «Por contrapedido»	Rótulo correcto, \$61,70	Pasa
PF-05	Abrir por URL la ficha del producto oculto	No se muestra	«No encontramos este producto» (la API responde 404)	Pasa
Acceso y permisos (HU-02)				
PF-06	Entrar a /admin/productos sin sesión	Lleva a la pantalla de acceso	Redirige a /acceso	Pasa
PF-07	Iniciar sesión con contraseña incorrecta	Mensaje de error sin decir cuál dato está mal	«Correo o contraseña incorrectos.»	Pasa
PF-08	Cliente intenta entrar al panel	No puede entrar ni consultar la API del panel	Vuelve a la tienda; la API responde 403	Pasa
PF-09	Administrador inicia sesión	Entra directo al panel de productos	Panel con los 13 productos, también el oculto	Pasa
Consultar y filtrar (HU-03)				
PF-10	Vista de tabla	Código, nombre, categoría, precio, existencias y etiquetas	13 filas con sus etiquetas (contrapedido, existencias bajas, oculto, margen negativo)	Pasa
PF-11	Buscar «lego» y buscar por código «ps5-»	Coincidencias por nombre y por código, sin distinguir mayúsculas	3 sets de LEGO; PS5-CTL-DS y PS5-GOW-RAG	Pasa
PF-12	Filtros: categoría, margen negativo y no disponibles	Solo los productos que cumplen	5 de Trading Cards; con margen negativo solo los protectores; oculto solo el Funko limitado	Pasa
Ingresar (HU-01 y HU-03)				
PF-13	Importar un Excel con 1 fila válida y 3 con errores	Guarda la válida, rechaza las otras con su motivo y permite descargar el reporte	1 nuevo, 3 rechazadas: código vacío, costo no numérico y margen menor que -100 %; reporte descargado	Pasa
PF-14	Volver a importar la hoja de 13 productos	Actualiza por código y no duplica	0 nuevos, 13 actualizados; el total sigue en 14	Pasa
PF-15	Agregar un producto con el formulario (código en minúsculas)	Se guarda con el código normalizado y el precio calculado	LEG-21348 , precio final \$487,31 (300 → 345 → 431,25 → 487,31)	Pasa
PF-16	Agregar un producto con un código que ya existe	Lo rechaza y marca el campo código	«Ya existe un producto con el código PS5-GOW-RAG.»	Pasa
PF-17	Agregar un producto con margen de -100 %	No deja guardar	«El margen debe ser mayor que -100 %.»	Pasa
Modificar (HU-04) y flujo completo				
PF-18	Editar el margen de PS5-GOW-RAG de 20 % a 10 %	Recalcula y guarda el precio	Precio sin impuesto \$65,99 y final \$74,57; «Producto actualizado.»	Pasa

PF-19	El visitante abre Legos después de agregar LEG-21348	El producto nuevo aparece de inmediato	Aparece con \$487,31 y «Por contrapedido»	Pasa
PF-20	El visitante abre la ficha del producto editado	Muestra el precio nuevo	\$74,57	Pasa
PF-21	Excluir un producto	Se da de baja sin perder su historial	No implementado (§ 2.8)	Pendiente

4.4 Capturas de pantalla

Interfaz del cliente

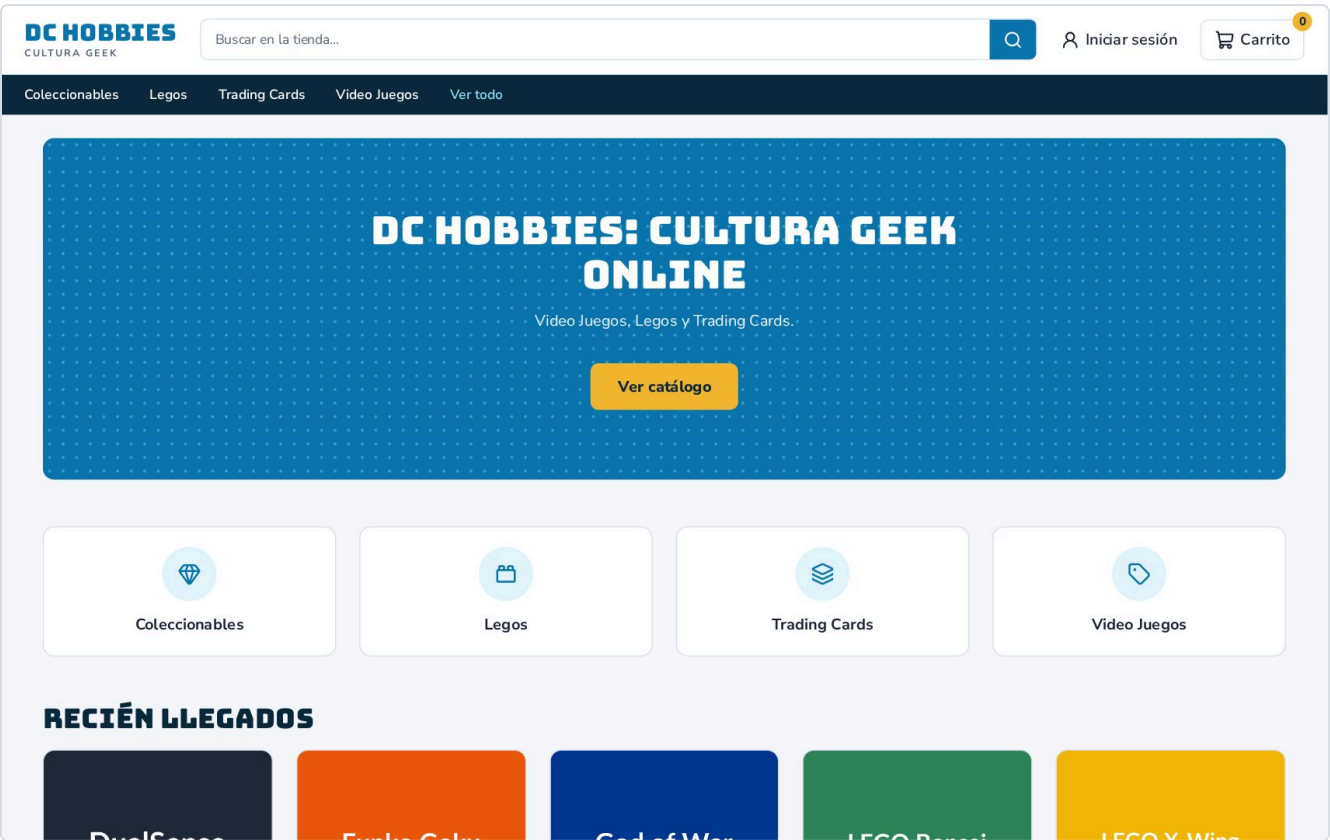


Figura 2. PF-01 · Página de inicio vista por un visitante, con las categorías que salen de la base de datos.

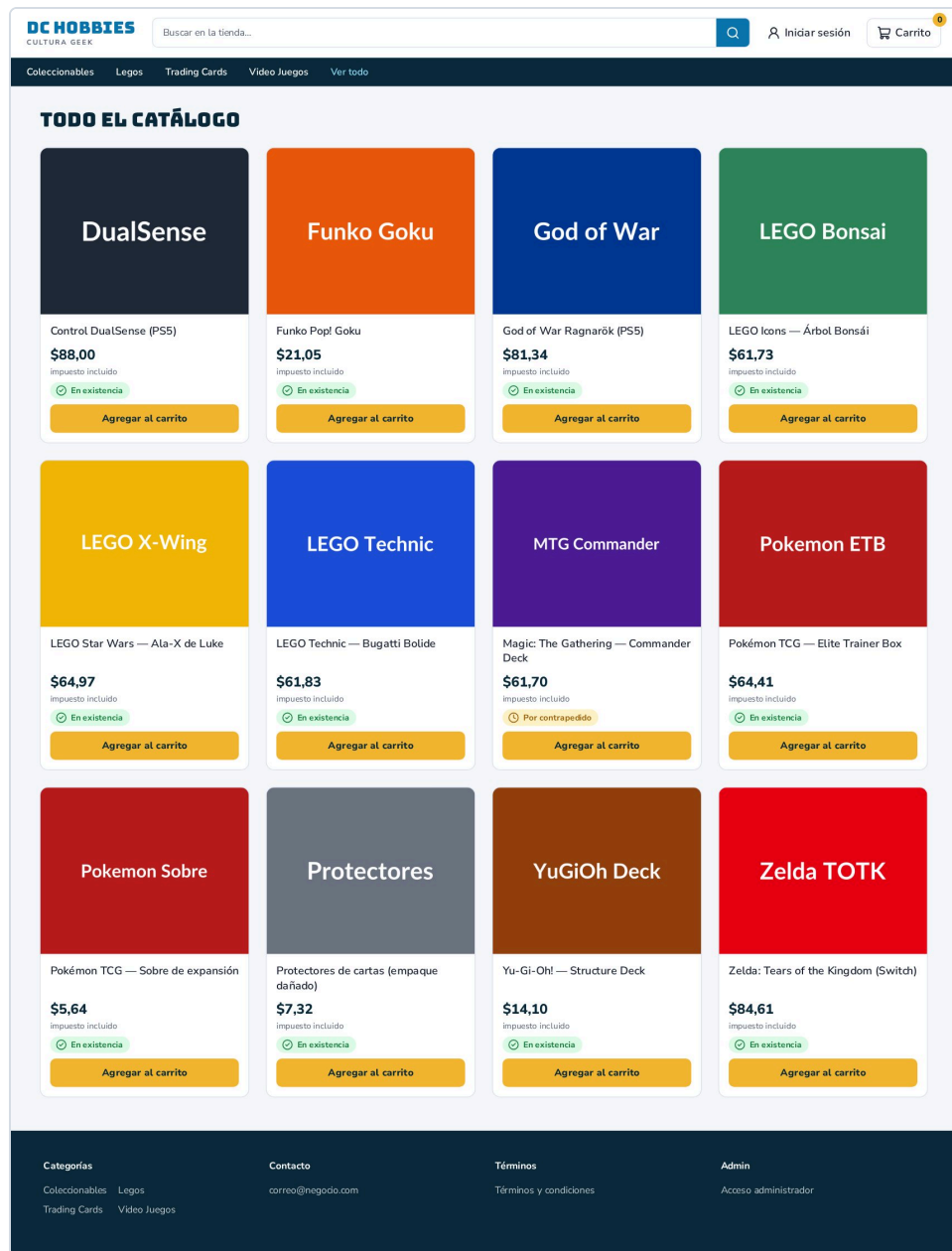


Figura 3. PF-02 · «Ver todo»: 12 productos visibles. El Funko de edición limitada (sin existencias ni contrapedido) no aparece y el mazo de Magic sale «Por contrapedido».

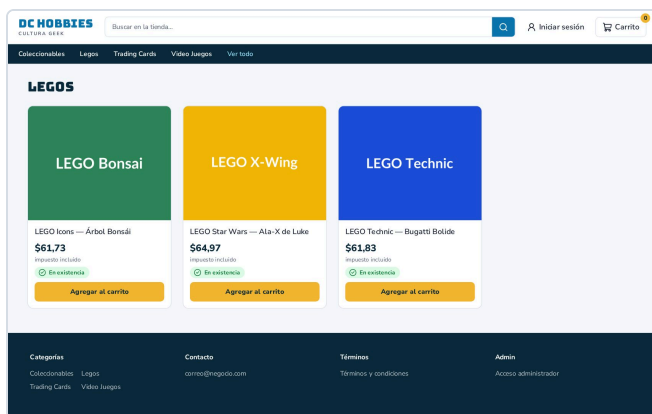


Figura 4. PF-03 · Catálogo de la categoría Legos.

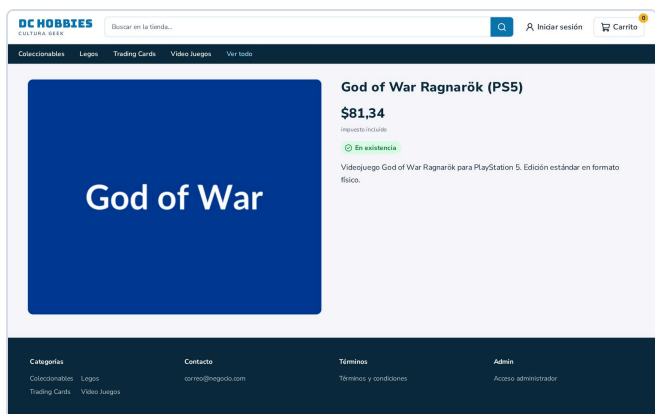


Figura 5. PF-04 · Ficha con existencias: \$81,34, igual que el Excel.

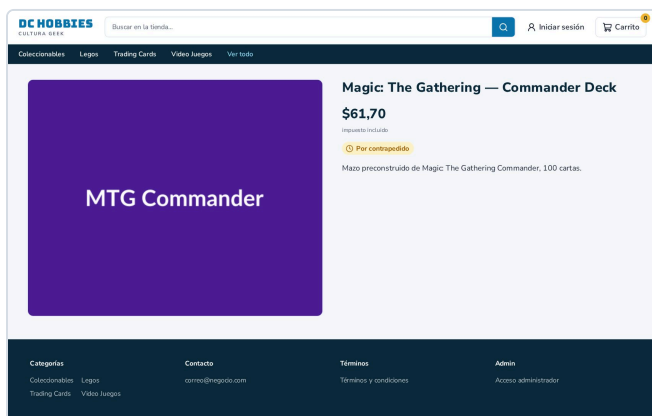


Figura 6. PF-04b · Ficha de un producto por contrapedido.

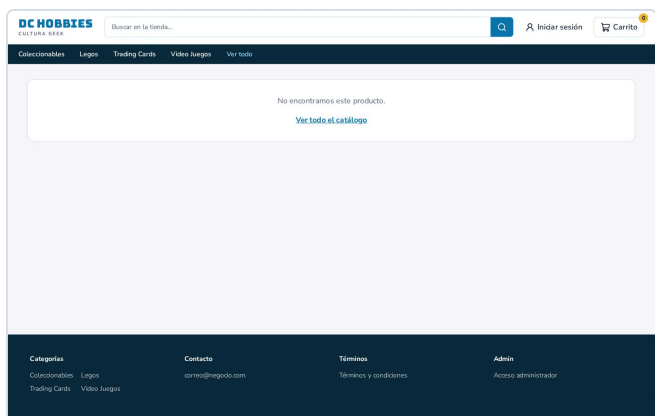


Figura 7. PF-05 · La ficha del producto oculto no se muestra ni por URL.

Acceso y permisos

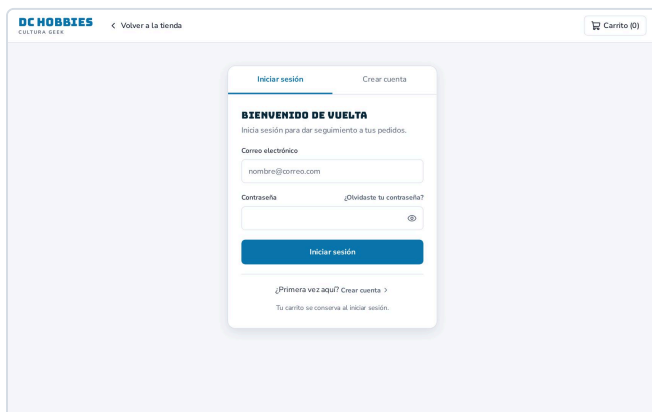


Figura 8. PF-06 · Sin sesión, /admin/productos lleva a la pantalla de acceso.

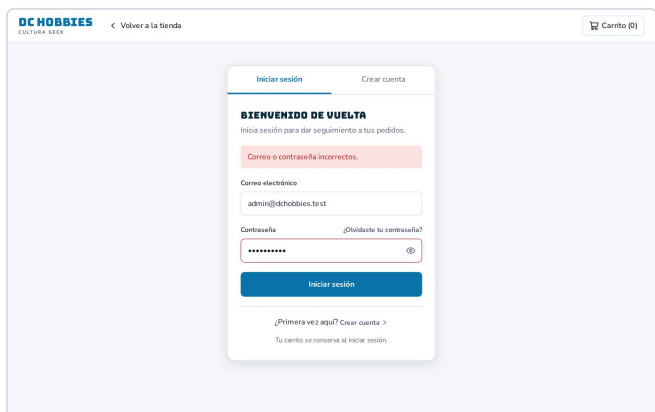


Figura 9. PF-07 · Contraseña incorrecta.

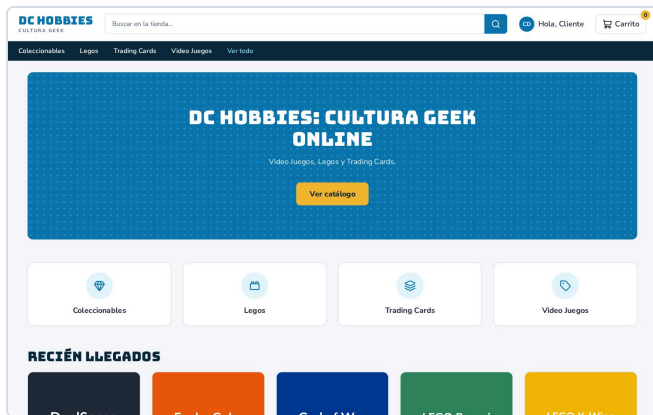


Figura 10. PF-08 · Con sesión de cliente, el panel lo devuelve a la tienda (y la API responde 403).

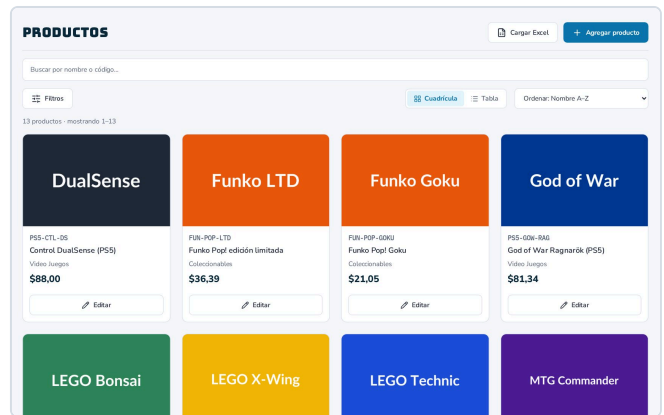


Figura 11. PF-09 · El administrador entra directo al panel de productos.

Consultar y filtrar

Buscar por nombre o código...					
Filtros Cuadrícula Tabla Ordenar: Nombre A-Z					
13 productos - mostrando 1-13					
CÓDIGO	NOMBRE	CATEGORÍA	PRECIO	EXISTENCIAS	ETIQUETAS
P55-CTL-DS	Control DualSense (P55)	Video Juegos	\$88,00	5	Por contrapedido Editar
FUN-POP-LTD	Funko Pop! edición limitada	Coleccionables	\$36,39	0	Existencias bajas Oculto en la tienda Editar
FUN-POP-GOKU	Funko Pop! Goku	Coleccionables	\$21,05	2	Por contrapedido Existencias bajas Editar
P55-GOW-RAG	God of War Ragnarök (P55)	Video Juegos	\$81,34	8	Editar
LEG-10281	LEGO Icons — Árbol Bonsái	Legos	\$61,73	4	Editar
LEG-75301	LEGO Star Wars — Ala-X de Luke	Legos	\$64,97	7	Editar
LEG-42151	LEGO Technic — Bugatti Bolide	Legos	\$61,83	5	Editar
MTG-CMD-001	Magic: The Gathering — Commander Deck	Trading Cards	\$61,70	0	Por contrapedido Existencias bajas Editar
PKM-SV-ETB	Pokémon TCG — Elite Trainer Box	Trading Cards	\$64,41	9	Editar
PKM-SV-SOB	Pokémon TCG — Sobre de expansión	Trading Cards	\$5,64	48	Editar
TCG-SLV-DAN	Protectores de cartas (empaquete dañado)	Trading Cards	\$7,32	6	Margen negativo Editar
YGO-SD-001	Yu-Gi-Oh! — Structure Deck	Trading Cards	\$14,10	11	Editar
NSW-ZEL-TOTK	Zelda: Tears of the Kingdom (Switch)	Video Juegos	\$84,61	6	Editar

Figura 12. PF-10 · Vista de tabla con existencias y etiquetas. Se ven los casos preparados: contrapedido, existencias bajas, oculto en la tienda y margen negativo.

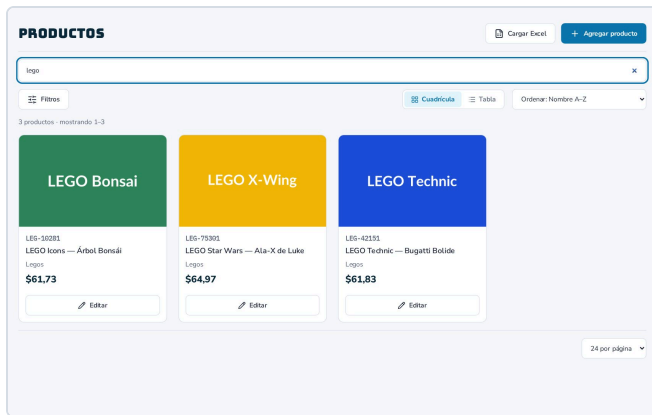


Figura 13. PF-11 · Búsqueda por nombre («lego»).

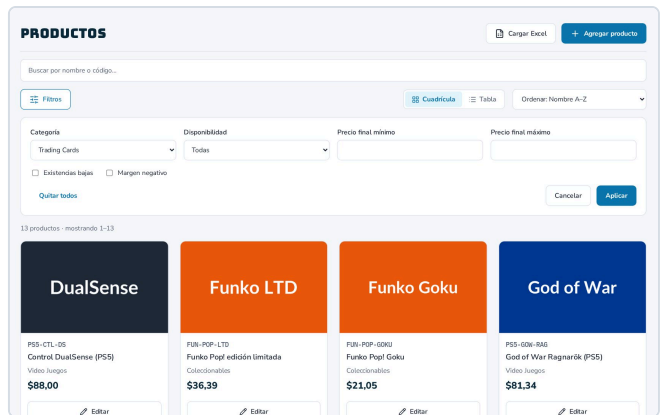


Figura 14. PF-12 · Panel de filtros abierto.

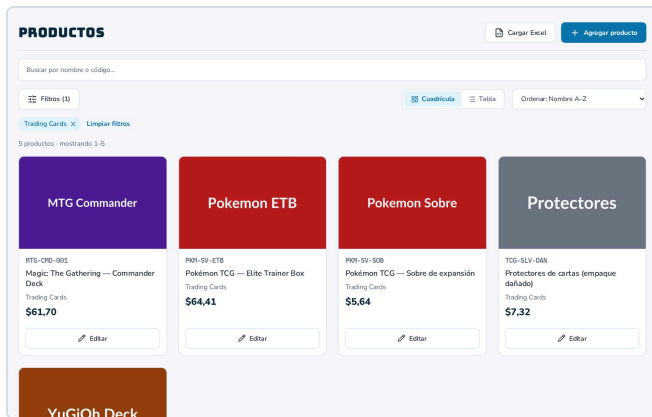


Figura 15. PF-12 · Filtro por categoría Trading Cards.

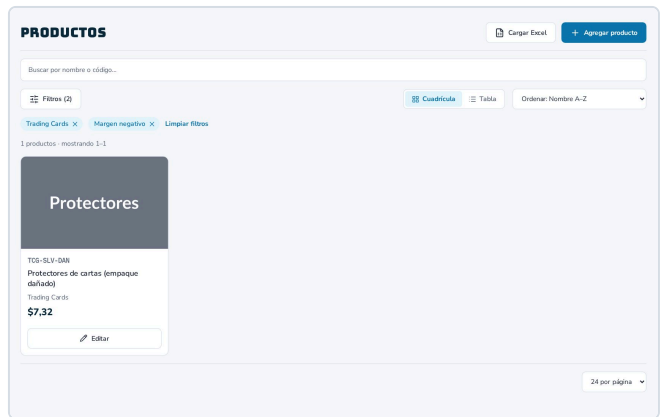


Figura 16. PF-12 · Trading Cards con margen negativo.

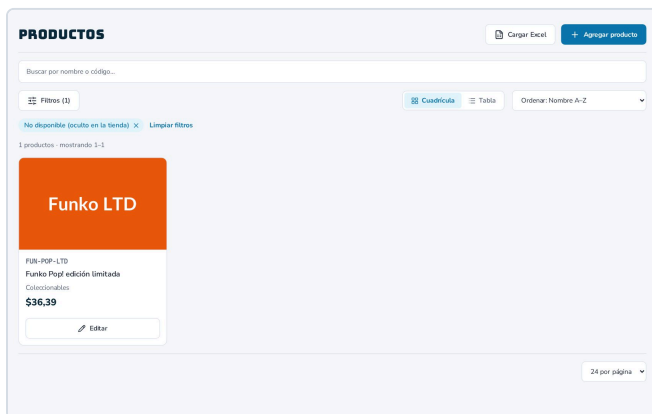


Figura 17. PF-12 · Filtro «No disponible (oculto en la tienda)».

Ingresar productos

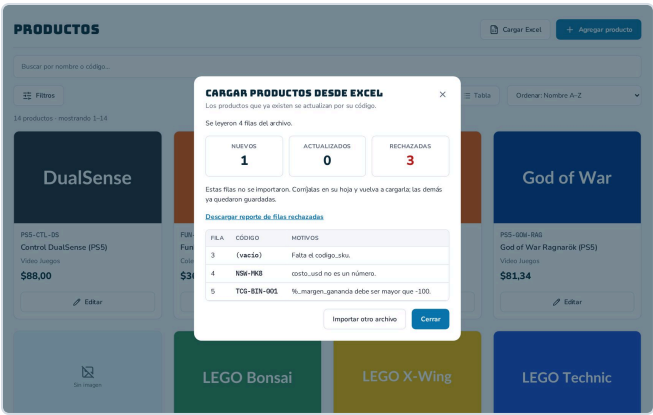


Figura 18. PF-13 · Importación con filas inválidas: se guarda la válida y se reportan las demás con su motivo.

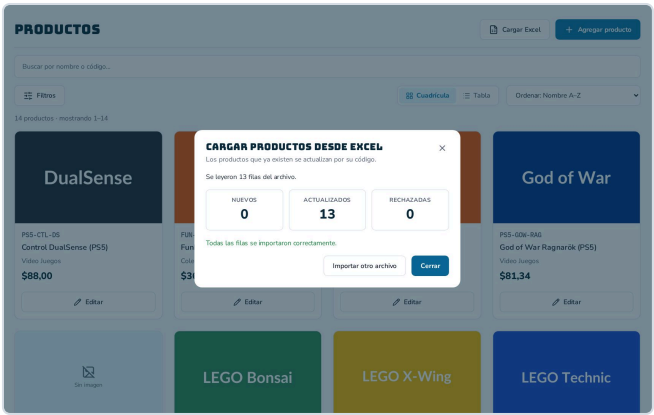


Figura 19. PF-14 · Reimportar la misma hoja: 13 actualizados y ningún duplicado.

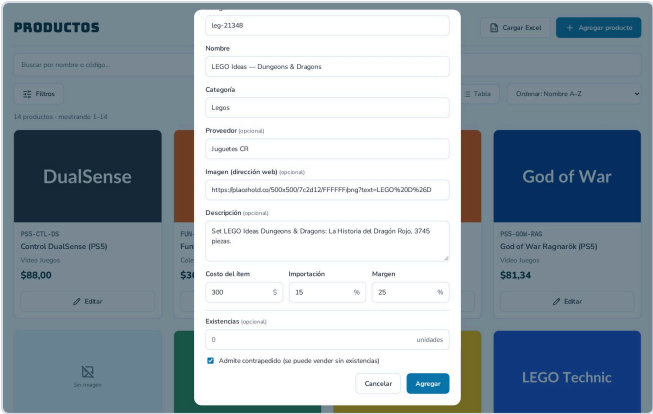


Figura 20. PF-15 · Formulario «Agregar producto» lleno (código escrito en minúsculas).

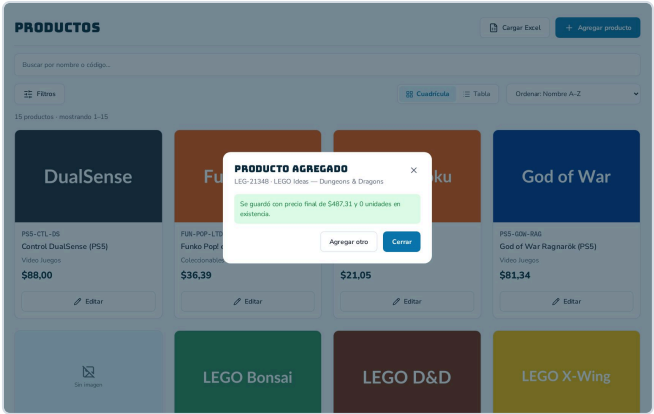


Figura 21. PF-15 · Producto guardado como LEG-21348 con el precio final calculado.

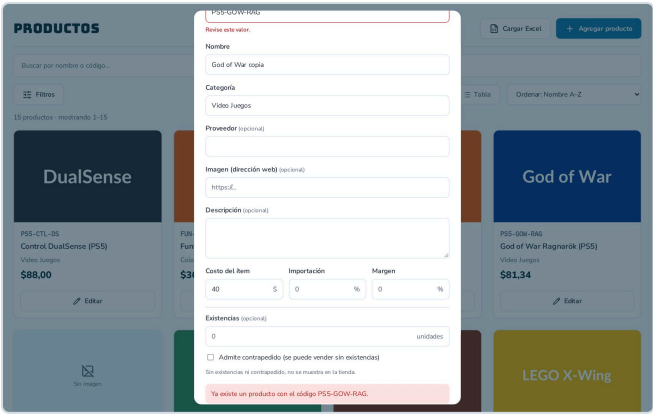


Figura 22. PF-16 · Código repetido: se rechaza y se marca el campo.

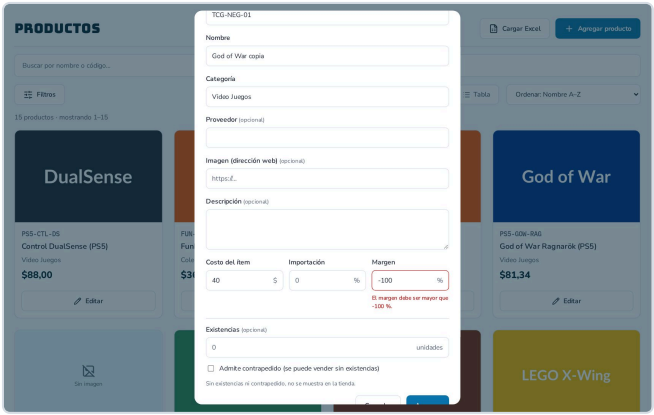


Figura 23. PF-17 · Margen de -100 %: no deja guardar.

Modificar el precio y verlo en la tienda

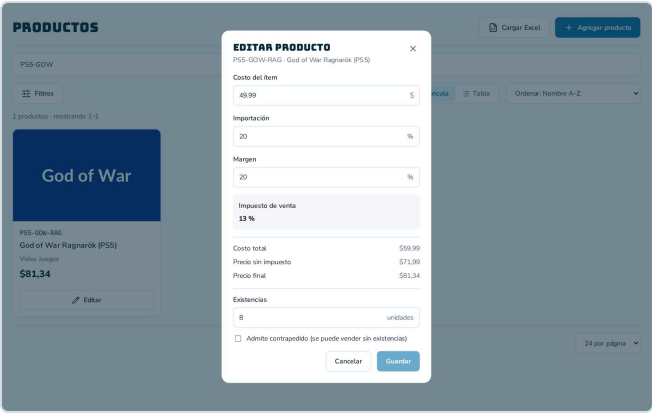


Figura 24. PF-18 · «Editar producto» antes del cambio (margen 20 %, \$81,34).

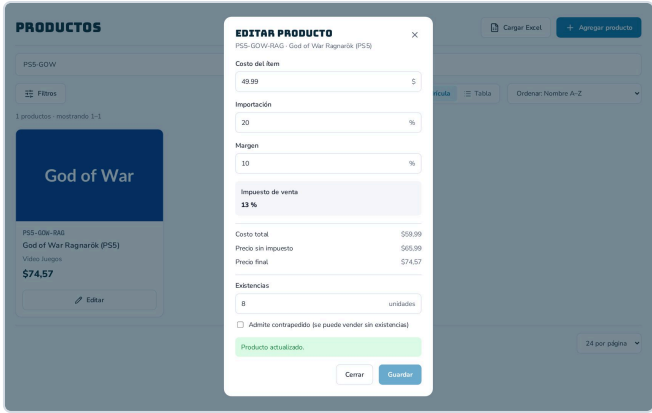


Figura 25. PF-18 · Después de guardar con margen 10 %: \$74,57.

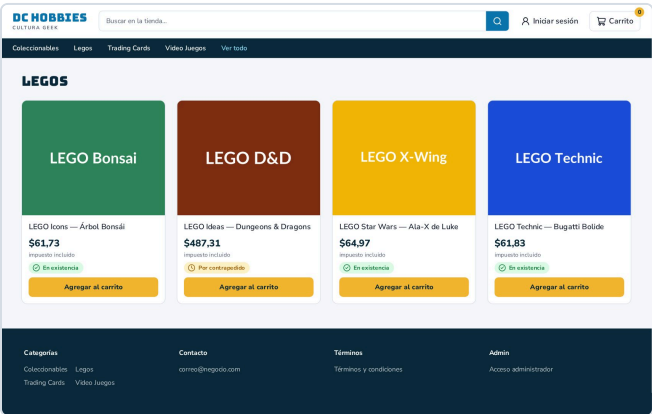


Figura 26. PF-19 · El visitante ya ve en Legos el producto que el administrador acaba de agregar.

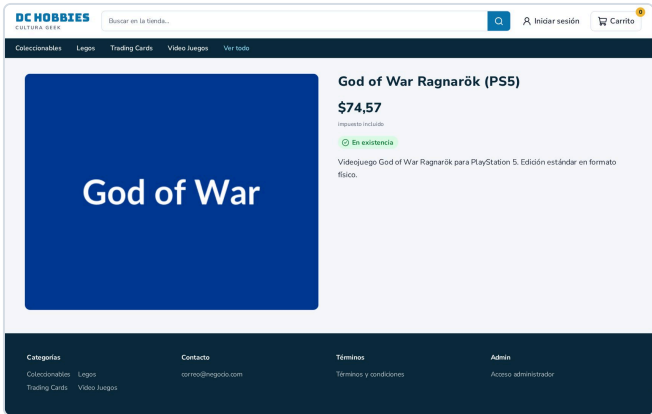


Figura 27. PF-20 · La ficha muestra el precio nuevo después de la edición.

4.5 Pruebas de la API

Como la protección y las validaciones reales están en el servidor, también probamos la API directamente con `curl`, sin pasar por la interfaz. Estas son las respuestas que obtuvimos:

ID	Petición	HTTP	Respuesta del servidor
API-01	GET /catalogo/admin/productos sin sesión	401	NO_AUTENTICADO : «Se requiere iniciar sesión.»
API-02	El mismo listado con sesión de cliente	403	NO_AUTORIZADO : «No tiene permiso para realizar esta acción.»
API-03	POST /catalogo/productos con sesión de cliente	403	NO_AUTORIZADO
API-04	Crear producto sin código ni nombre, costo -5 y margen -100	400	ENTRADA_INVALIDA con los cuatro mensajes juntos y campos : [sku, nombre, costoItem, margenGanancia]
API-05	Crear producto con código " ps5-gow-rag "	409	REGLA_DE_NEGOCIO_VIOLADA : «Ya existe un producto con el código PS5-GOW-RAG.» (regla RF-01)

API-06	PATCH /catalogo/productos/PS5-GOW-RAG con margen-100	400	«El margen tiene que ser un número mayor que -100 %.»
API-07	El mismo PATCH enviando precioFinal	400	«Solo se pueden cambiar costoItem, porcentajeImportacion, margenGanancia, existencias, admiteContrapedido; no se aceptan: precioFinal.» y pide al menos uno de esos campos
API-08	PATCH a un SKU que no existe	404	RECURSO_NO_ENCONTRADO
API-09	GET /catalogo/productos/FUN-POP-LTD (oculto)	404	El catálogo público no lo devuelve
API-10	GET /catalogo/productos?limite=48	200	12 productos: 11 en_existencia y 1 por_contrapedido; el oculto no está
API-11	POST /admin/sesion con contraseña incorrecta	401	CREDENCIALES_INVALIDAS : «Correo o contraseña incorrectos.»

5. Sprint Backlog del Sprint 1

El sprint se manejó en Jira, en el tablero «**Sprint 1 — Productos**» (22 de setiembre al 7 de octubre de 2026). Partimos cada historia en tareas pequeñas marcadas por capa: **[BD]** base de datos, **[BE]** servidor, **[FE]** cliente web y **[Docs]** documentación. Cada tarea se trabajó en su propia rama y entró a `develop` por un Pull Request (hasta el 4 de octubre se integraron 41).

Tabla 3. Resumen del sprint

Historia	Puntos	Tareas	Terminadas	Estado de la historia
HU-01 Importación del catálogo desde Excel	8	7	7	Terminada
HU-02 Inicio de sesión con permisos por rol	5	8	8	Parcial
HU-03 Registro y clasificación de productos	5	10	10	Parcial
HU-04 Cálculo automático del precio de venta	8	6	6	Terminada
HU-05 Consulta del catálogo con disponibilidad real	8	12	12	Terminada
Fundaciones técnicas (sin puntos)	—	7	7	Terminada
Cierre del sprint: documentación y evidencia	—	5	3	En curso
Total	34	55	53	24 de 34 puntos completos

Todas las tareas de HU-02 y HU-03 se terminaron, pero las historias quedan parciales porque tienen criterios de aceptación que no se cubrieron con esas tareas (recuperación de contraseña y discontinuar un producto, ver sección 7). Por eso no contamos sus puntos como completos.

5.1 Tareas del sprint

Clave	Tarea	Responsable	Estado
HU-01 • Importación del catálogo desde Excel (SCRUM-44)			
SCRUM-30	[BE] Plantilla oficial de Excel para importar productos	Jeferson Marín	Hecha
SCRUM-31	[FE] Pantalla de importación de Excel	Jeferson Marín	Hecha
SCRUM-32	[BE] Endpoint para subir el Excel de productos	Jeferson Marín	Hecha
SCRUM-33	[BE] Leer las filas del Excel de productos	Jeferson Marín	Hecha
SCRUM-34	[BE] Validar cada fila del Excel	Jeferson Marín	Hecha
SCRUM-35	[BE] Guardar la importación en una transacción	Jeferson Marín	Hecha

SCRUM-36	[BE] Reporte descargable de filas rechazadas	Jeferson Marín	Hecha
HU-02 · Inicio de sesión con permisos por rol (SCRUM-3)			
SCRUM-5	[BD] Tablas de usuarios y roles	Joaquín Rodríguez	Hecha
SCRUM-6	[Docs] Matriz de permisos por rol	Juan Loaiza	Hecha
SCRUM-7	[BE] Hash seguro de contraseñas	Juan Loaiza	Hecha
SCRUM-9	[BE] Iniciar sesión con cookie de sesión	Juan Loaiza	Hecha
SCRUM-10	[BE] Middlewares de sesión y autorización por rol	Jeferson Marín	Hecha
SCRUM-11	[FE] Pantalla de inicio de sesión	Juan Loaiza	Hecha
SCRUM-12	[FE] Rutas del panel protegidas y header según la sesión	Juan Loaiza	Hecha
SCRUM-70	[BE] Consultar la sesión actual	Juan Loaiza	Hecha
HU-03 · Registro y clasificación de productos (SCRUM-42)			
SCRUM-14	[BD] Tablas de categorías, subcategorías y productos <i>(con el EER corregido quedó solo Producto, C-04)</i>	Joaquín Rodríguez	Hecha
SCRUM-15	[BD] Categorías y subcategorías iniciales	Joaquín Rodríguez	Hecha
SCRUM-16	[BE] Listar categorías y subcategorías	Jeferson Marín	Hecha
SCRUM-18	[BE] Crear producto con código único	Jeferson Marín	Hecha
SCRUM-21	[BE] Listado administrativo de productos	Joaquín Rodríguez	Hecha
SCRUM-28	[FE] Modal de agregar y editar producto	Agustín Soto	Hecha
SCRUM-61	[FE] Pantalla de Productos del administrador	Agustín Soto	Hecha
SCRUM-71	[FE] Panel de filtros de Productos	Agustín Soto	Hecha
SCRUM-72	[FE] Vista de tabla y estados vacíos de Productos	Agustín Soto	Hecha
SCRUM-73	[FE] Conectar la pantalla de Productos a la API	Agustín Soto	Hecha
HU-04 · Cálculo automático del precio de venta (SCRUM-43)			
SCRUM-19	[BE] Editar costo, importación y margen del producto	Joaquín Rodríguez	Hecha
SCRUM-22	[Docs] Documentar la fórmula de precio con el PO	Agustín Soto	Hecha
SCRUM-23	[BD] Campos de costo, importación y margen del producto	Joaquín Rodríguez	Hecha
SCRUM-24	[BE] Parámetros de precio en la configuración	Jeferson Marín	Hecha
SCRUM-25	[BE] Motor de precios	Jeferson Marín	Hecha
SCRUM-26	[BE] Pruebas unitarias del motor de precios	Jeferson Marín	Hecha
HU-05 · Consulta del catálogo con disponibilidad real (SCRUM-45)			
SCRUM-38	[BD] Tablas de existencias y movimientos de inventario <i>(con el EER corregido: stock y producto_administra, C-11)</i>	Joaquín Rodríguez	Hecha
SCRUM-39	[BE] Disponibilidad de cada producto	Jeferson Marín	Hecha
SCRUM-40	[BE] Catálogo público con paginación	Jeferson Marín	Hecha

SCRUM-49	[BE] Ocultar productos sin existencias ni contrapedido	Jeferson Marín	Hecha
SCRUM-68	[BE] Ficha de producto	Jeferson Marín	Hecha
SCRUM-50	[FE] Ficha de detalle del producto	Juan Loaiza	Hecha
SCRUM-51	[FE] Página de catálogo por categoría	Juan Loaiza	Hecha
SCRUM-60	[FE] Página de inicio de la tienda	Juan Loaiza	Hecha
SCRUM-67	[FE] Header, Footer, ProductCard y StoreLayout	Juan Loaiza	Hecha
SCRUM-69	[FE] Cliente HTTP y hook useCatalogo	Juan Loaiza	Hecha
SCRUM-74	[FE] Conectar la página de inicio a la API	Juan Loaiza	Hecha
SCRUM-75	[FE] Aplicar la identidad visual elegida por el cliente	Juan Loaiza	Hecha
Fundaciones técnicas (SCRUM-58)			
SCRUM-59	[BD] Esquema por módulo, migraciones y scripts de la base de datos	Joaquín Rodríguez	Hecha
SCRUM-8	[BD] Datos semilla de referencia y de prueba	Joaquín Rodríguez	Hecha
SCRUM-62	[BD] Pruebas de restricciones de la base de datos	Joaquín Rodríguez	Hecha
SCRUM-63	[BE] Estructura base del servidor	Jeferson Marín	Hecha
SCRUM-64	[BE] Módulo catálogo de referencia y esqueletos de los demás módulos	Jeferson Marín	Hecha
SCRUM-65	[FE] Proyecto React + Vite con React Router	Juan Loaiza	Hecha
SCRUM-66	[Docs] Documento de arquitectura y modelo de datos	Juan Loaiza	Hecha
Cierre del Sprint 1: documentación y evidencia (SCRUM-46)			
SCRUM-52	[BD] Regenerar el script DDL y los datos de prueba	Joaquín Rodríguez	En revisión
SCRUM-53	[BD] Diagrama ER actualizado	Joaquín Rodríguez	Hecha
SCRUM-54	[Docs] Pruebas funcionales con evidencia	Joaquín Rodríguez	Hecha
SCRUM-55	[Docs] Documento del Sprint 1 (este documento)	Juan Loaiza	En curso
SCRUM-56	[Docs] Retrospectiva del Sprint 1	Juan Loaiza	Hecha

La demostración funcional (flujo administrador → cliente) es un entregable del cierre del sprint; el recorrido de los casos PF-15 a PF-20 es el que vamos a mostrar.

5.2 Tareas que quedaron obsoletas

Estas tareas se crearon en la planificación pero las sacamos del sprint, sobre todo por el cambio al EER corregido o porque su trabajo quedó cubierto por otra tarea. Las dejamos en Jira marcadas como *[OBSOLETA]* para no perder el historial.

Clave	Tarea	Motivo
SCRUM-13	Cierre de sesión y expiración	No corresponde a ningún RF; se reevalúa con RF-54 en el Sprint 2.
SCRUM-17	Validación del código/SKU	Quedó dentro de SCRUM-18 (crear producto con código único).
SCRUM-20	Formulario de registro en UI	Lo reemplazó el modal de agregar producto (SCRUM-28).

SCRUM-27	Integrar el cálculo al guardar	El precio ya no se guarda: se calcula al consultar (C-01).
SCRUM-29	Precio manual (override)	El precio siempre lo calcula el motor; el ajuste se hace con el margen (RN-02).
SCRUM-37	Restringir la importación por rol	Quedó cubierto por los middlewares (SCRUM-10) en el endpoint de SCRUM-32.
SCRUM-41	Alerta de existencias bajas en el panel	Pasa al Sprint 2 con los movimientos de inventario; en el listado sí quedó la etiqueta.
SCRUM-47	Reimportación: actualizar productos existentes por código	Quedó dentro del guardado en transacción (SCRUM-35), con <code>ON CONFLICT (sku)</code> .
SCRUM-48	Calcular y persistir el precio al importar	El precio no se persiste (C-01).

6. Product Backlog actualizado

Partimos del Product Backlog inicial del Sprint 0 y le agregamos el estado de cada elemento después de este sprint. Al final están los elementos nuevos que salieron de la entrevista del 24 de setiembre (todos aprobados por el cliente); como todavía no los estimamos, la estimación y el sprint quedan por definir en la próxima planificación.

ID	Historia / requerimiento	Prioridad	Est.	Sprint	Estado después del Sprint 1
Planeadas para el Sprint 1					
HU-01	Importación del catálogo desde Excel	Alta (M)	8	S1	Terminada Las existencias entran por HU-07.
HU-02	Inicio de sesión con permisos por rol	Alta (M)	5	S1 → S2	Parcial Falta la recuperación de contraseña (RF-54).
HU-03	Registro y clasificación de productos	Alta (M)	5	S1 → S2	Parcial Faltan discontinuar (RF-18) y editar los datos generales; la subcategoría se descartó (C-04).
HU-04	Cálculo automático del precio de venta	Alta (M)	8	S1	Terminada INC-05 cerrada con el Excel del cliente.
HU-05	Consulta del catálogo con disponibilidad real	Alta (M)	8	S1	Terminada Falta el total para paginar «Ver todo».
Planeadas para el Sprint 2					
HU-07	Ingreso de mercancía en una sola pantalla	Alta (M)	5	S2	Pendiente. Incluye los movimientos de inventario.
HU-08	Alerta de existencias bajas	Alta (M)	5	S2	Pendiente. La etiqueta del listado ya existe.
HU-09	Registro de ventas de otros canales	Alta (M)	5	S2	Pendiente. Hay que definir con el cliente si una venta lleva varios productos (aprobado #5).
HU-10	Carrito persistente con contrapedido	Alta (M)	8	S2	Pendiente. El botón y el contador de la tienda dependen de esta historia.
HU-11	Registro del cliente en su primera compra	Alta (M)	5	S2	Pendiente. Por confirmar si «Crear cuenta» está antes de comprar.
PB-01	Gestión de precios, descuentos y márgenes (RF-51)	Alta (M)	5	S2	Avanzada: la edición de margen, costo e importación ya existe; faltan los descuentos (ofertas, C-08).
PB-04	Canal de contacto al correo del negocio (RF-56)	Alta (M)	2	S2	Pendiente.
Planeadas para el Sprint 3					
HU-12	Confirmación del pedido, pago y facturación	Alta (M)	13	S3	Pendiente. Falta definir la referencia de un pago sin respuesta de la pasarela.
HU-13	Descuento por nivel de cliente frecuente	Alta (M)	8	S3	Pendiente. Ahora se apoya en la entidad Oferta.
HU-14	Seguimiento y cancelación de pedidos	Alta (M)	5	S3	Pendiente. La máquina de estados ya tiene pruebas.
HU-15	Reportes de ventas, inventario y pedidos por cliente	Alta (M)	8	S3	Pendiente. Las vistas de reportes ya existen en la BD.
PB-02	Selección de modalidad de entrega (RF-62)	Alta (M)	5	S3	Pendiente. Faltan las tarifas (aprobado #11).

HU-06	Búsqueda y filtrado de productos (cliente)	Media (S)	5	S3	Pendiente. Incluye el buscador de la barra superior.
PB-03	Pedidos activos simultáneos sin límite (RF-29)	Alta (M)	2	S3	Pendiente.
Por priorizar y fuera del alcance inicial (sin cambios)					
PB-05	Bitácora de auditoría (RF-52)	Media (S)	5	—	Por definir dónde se guarda (§ 3.4).
PB-06 a PB-10	Historial del cliente, reposición, retiro de temporada, importación de clientes, número de guía	Media (S)	3-5	—	Por priorizar. RF-09 (temporada) y el histórico de costos (RF-14) necesitan un cambio al EER.
PB-11 a PB-15	Métodos de pago adicionales, contra entrega, notificación por correo, cupones, respaldo automático	Baja / Muy baja	3-5	—	Fuera del alcance inicial.
Nuevos (entrevista del 24/09/2026, aprobados por el cliente)					
PB-16	Página de inicio con vitrina y productos por categoría (#8, #16)	Media	—	S1	Terminada Falta ordenar «Recién llegados» por fecha.
PB-17	Filtros del catálogo en el panel del administrador (#9)	Media	—	S1	Terminada
PB-18	Cuentas para empleados (#6)	—	—	S2	Pendiente. Por definir qué puede hacer un empleado.
PB-19	Cerrar sesión y reglas de contraseña (#15)	—	—	S2	Pendiente. El cliente pidió de 8 a 12 caracteres con una mayúscula, un número y un carácter especial.
PB-20	Lista de clientes en el panel (#2)	—	—	—	Por estimar y priorizar.
PB-21	Corregir los datos de un cliente (#3)	—	—	—	Por estimar y priorizar.
PB-22	Últimos pedidos del cliente desde su ficha (#1)	—	—	—	Por estimar y priorizar.
PB-23	Buscar y filtrar pedidos en el panel (#4)	—	—	—	Por estimar y priorizar.
PB-24	Tablero de inicio del administrador (#7)	—	—	—	Por estimar y priorizar.
PB-25	Editar la escala de niveles y los descuentos (#10)	—	—	—	Por estimar. Ligado a HU-13 y a la entidad Oferta.
PB-26	Configurar las tarifas de envío (#11)	—	—	—	Por estimar. Ligado a PB-02.
PB-27	Registrar los datos fiscales del negocio (#12)	—	—	—	Por estimar. Ligado a HU-12.
PB-28	Editar el texto de términos y condiciones (#13)	—	—	—	Por estimar.
PB-29	Umbral de alerta de existencias configurable (#14)	—	—	—	Por confirmar con el cliente; hoy es fijo en 2 unidades (RN-04).

7. Historias de usuario y criterios de aceptación abordados

Estas son las historias que trabajamos, con sus criterios de aceptación tal como quedaron en el Sprint 0 (formato Gherkin) y el estado de cada uno. Cuando un criterio se tuvo que ajustar por un cambio de la sección 3, lo indicamos.

HU-01 — Importar el catálogo desde Excel Terminada

Como administrador, quiero cargar mi hoja de Excel de productos, para dejar de mantener el inventario a mano. · EP-08 · RF-57, RF-58, RF-59 · 8 puntos

Criterio	Estado	Evidencia y notas
Dado que tengo mi hoja de Excel con código, categoría, subcategoría, nombre, descripción, costo, importación, margen y cantidad Cuando la cargo en el sistema Entonces los productos quedan registrados con su precio ya calculado Y el sistema me informa cuántas filas importó	Cumplido Ajustado	PF-14. Se usan las columnas reales de la hoja del cliente, que no trae subcategoría (C-04) ni cantidad: las existencias se registran aparte (§ 2.8).
Dado que la hoja contiene filas con código vacío o costo no numérico Cuando ejecuto la importación Entonces el sistema importa las filas válidas Y me entrega un reporte descargable con cada fila rechazada y su motivo	Cumplido	PF-13, pruebas <code>importacion-de-excel</code> y <code>reporte-de-rechazos</code> .
Dado que ya importé la hoja previamente Cuando vuelvo a importarla Entonces el sistema actualiza los productos existentes por código Y no crea registros duplicados	Cumplido	PF-14 (0 nuevos, 13 actualizados) y prueba «RF-59: reimportar la misma hoja...».

HU-02 — Iniciar sesión con permisos por rol Parcial

Como usuario del sistema, quiero iniciar sesión con mi cuenta, para acceder únicamente a las funciones que corresponden a mi rol. · EP-07 · RF-49, RF-50, RF-53, RF-54 · 5 puntos

Criterio	Estado	Evidencia y notas
Dado que existe la cuenta única de administrador creada al poner el sistema en marcha Cuando inicio sesión con sus credenciales Entonces accedo al panel de administración completo	Cumplido	PF-09. La BD solo permite un administrador (índice único, RF-49).
Dado que tengo rol Cliente Cuando intento abrir una pantalla del panel de administración Entonces el sistema me deniega el acceso	Cumplido	PF-08, API-02 y API-03 (403).
Dado que olvidé mi contraseña Cuando solicito recuperarla Entonces recibo un enlace de restablecimiento de un solo uso y con vencimiento	Pendiente	Pasa al Sprint 2 (RF-54 es <i>Should</i>). El enlace «¿Olvidaste tu contraseña?» está deshabilitado.

HU-03 — Registrar y clasificar productos Parcial

Como administrador, quiero registrar mis productos con su código, categoría y subcategoría, para tener un catálogo estructurado y sin duplicados. · EP-01 · RF-01, RF-02, RF-06, RF-18 · 5 puntos

Criterio	Estado	Evidencia y notas
Dado que voy a dar de alta un producto nuevo Cuando completo código, categoría, subcategoría, nombre, descripción, costo, importación, margen, cantidad, marca de contrapedido, proveedor e imagen Entonces el producto queda guardado Y aparece en el catálogo bajo su categoría y subcategoría	Cumplido Ajustado	PF-15 y PF-19. Sin subcategoría (C-04). La cantidad es opcional: si no se indica, entra con 0 existencias y aparece en su categoría si admite contrapedido o cuando se le registre mercancía.
Dado que ya existe un producto con el código PKM-001 Cuando intento guardar otro con ese mismo código Entonces el sistema rechaza el guardado Y me indica que el código está duplicado	Cumplido	PF-16, API-05 y la prueba «RF-01: registrar PKM-001 dos veces...». También detecta el duplicado si se escribe en minúsculas o con espacios.
Dado que un producto deja de venderse Cuando lo marco como discontinuado Entonces desaparece del catálogo público Y conserva su historial de movimientos y ventas	Pendiente	Es la operación «Excluir». Primer punto del Sprint 2 (§ 2.8).

HU-04 — Calcular el precio de venta automáticamente Terminada

Como administrador, quiero que el sistema calcule el precio de venta a partir del costo, la importación y el margen, para no volver a calcular precios a mano en Excel. · EP-01 · RF-03, RF-04, RF-05, RF-43 · 8 puntos

Criterio	Estado	Evidencia y notas
Dado un producto con costo de ítem ₡100 y 20 % de importación Cuando el sistema calcula su costo total Entonces el costo total es ₡120	Cumplido	Prueba del motor de precios. Ahora los montos van en dólares (C-03): \$100 → \$120.
Dado ese costo total de ₡120 y un margen de 25 % Cuando el sistema calcula el precio Entonces el precio sin impuesto es ₡150 Y el precio final mostrado al cliente es ₡169,50 tras agregar el 13 % de impuesto	Cumplido	Prueba «RF-03, RF-04, RF-05...» y el caso real del Excel en PF-04 (\$81,34). PF-18 muestra el recálculo al editar.
Dado un producto defectuoso o de liquidación Cuando le asigno un margen de -10 % Entonces el sistema lo acepta Y calcula un precio por debajo del costo	Cumplido	Prueba «RN-02: un margen negativo vende por debajo del costo» y TCG-SLV-DAN en PF-12. Un margen de -100 % o menos se rechaza (PF-17, API-06).

HU-05 — Consultar el catálogo con disponibilidad real Terminada

Como visitante, quiero ver el catálogo con la disponibilidad real de cada producto, para saber qué puedo comprar de inmediato y qué no. · EP-01 · RF-07, RF-08, RF-12 · 8 puntos

Criterio	Estado	Evidencia y notas
Dado que un producto tiene existencias Cuando abro su ficha Entonces veo su imagen, nombre, descripción y precio final Y veo el rótulo "con existencias"	Cumplido	PF-04 (el rótulo dice «En existencia»).
Dado que un producto tiene cero existencias pero admite contrapedido Cuando lo consulto Entonces el sistema lo muestra rotulado como disponible por contrapedido	Cumplido	PF-02, PF-04byPF-19.
Dado que un producto tiene cero existencias y no admite contrapedido Cuando navego el catálogo público Entonces el producto no aparece ni en el catálogo ni en los resultados de búsqueda	Cumplido	PF-02, PF-05, API-09 y API-10. La búsqueda del cliente es de HU-06, pero usa la misma consulta pública, que ya excluye estos productos.

Historias agregadas en el sprint (aprobadas el 24/09)

Estas dos no tenían criterios en el Sprint 0, así que los escribimos al planificarlas:

Criterio	Estado	Evidencia
PB-16 • Como visitante, quiero ver en la página de inicio los productos agrupados por categoría, para descubrir qué hay sin tener que buscar.		
Dado que entro a la página de inicio Cuando se carga Entonces veo una sección por cada categoría con productos visibles Y cada sección tiene un enlace para ver toda la categoría	Cumplido	PF-01
PB-17 • Como administrador, quiero filtrar mi catálogo por existencias bajas o margen negativo y ordenarlo por existencias, para revisar qué tengo que reponer o qué tengo en liquidación.		
Dado que estoy en el panel de productos Cuando aplico el filtro de margen negativo Entonces solo veo los productos con margen menor que 0 Y puedo quitar el filtro desde su etiqueta	Cumplido	PF-12

8. Retrospectiva del equipo

Hicimos la retrospectiva al cierre del sprint. La organizamos en las tres preguntas de siempre: qué funcionó, qué nos costó y qué vamos a cambiar.

Qué funcionó

- **Hacer primero un módulo de referencia.** Con el módulo de catálogo completo (SCRUM-64) como plantilla, los demás pudimos seguir la misma estructura sin discutirla cada vez.
- **Ramas y Pull Requests.** Trabajar cada tarea en su rama y unirla por PR nos dejó revisar los cambios antes de integrarlos y evitó conflictos grandes.
- **Pruebas desde el principio.** Las pruebas del motor de precios encontraron el problema del redondeo antes de que llegara a la interfaz.
- **Validar con datos reales.** Comparar contra el Excel del cliente cerró INC-05 y nos dio una plantilla de importación que el negocio puede usar tal cual.
- **Anotar lo que no entra.** El archivo `cambios-siguiente-sprint.md` nos ayudó a no meter trabajo fuera del alcance y a no olvidar lo que quedaba pendiente.
- **Hablar con el cliente a tiempo.** La entrevista del 24/09 nos dejó aprobados los requerimientos de los wireframes antes de construirlos.

Qué nos costó

- **Cambiar el modelo a mitad del sprint.** La propuesta de rediseño y la vuelta al EER corregido (§ 3.1) nos obligaron a rehacer parte del servidor, dejaron 9 tareas obsoletas y desactualizaron documentos de diseño.
- **Excluir quedó sin decidir.** Al volver al EER se perdió la columna de baja lógica, y no tomamos la decisión a tiempo.
- **Integración al final.** La conexión del panel con la API, el formulario, la edición, los filtros, la tabla y la identidad visual entraron todos en los últimos dos días, y quedó poco margen para probar el sistema completo.
- **La moneda.** Hasta el 1 de octubre supimos que todo va en dólares; las semillas todavía tienen montos pensados en colones.
- **Jira incompleto.** Muchas tareas no tienen descripción ni criterios, y las épicas siguen en «To Do» aunque sus tareas estén hechas.

Qué vamos a cambiar

- **Congelar el modelo al planificar.** Cualquier cambio después va como migración nueva, con su fila en `modelo-datos.md` y acordado por todo el equipo.
- **Decisiones de modelo primero.** RF-18 y los pendientes marcados como prioritarios se deciden el primer día del Sprint 2.
- **Integrar a mitad del sprint.** Meta: tener el flujo de punta a punta conectado a mitad del sprint y una demo interna dos días antes del cierre.
- **Documentar en el mismo PR.** Si un PR cambia el diseño, actualiza también el documento que lo describe.
- **Jira al día.** Cada tarea con una descripción corta y su criterio de terminado, y cerrar las épicas al terminar sus historias.

9. Pendientes para el Sprint 2

Estos son los puntos que quedan abiertos del Sprint 1, en el orden en que pensamos atacarlos. El detalle de cada uno está en `documentos/requerimientos/cambios-siguiente-sprint.md`.

#	Pendiente	Qué falta
1	Excluir un producto (RF-18)	Decidir entre las opciones de § 2.8 (proponemos la columna <code>estado</code>), migración 010, ruta para discontinuar y filtro <code>descontinuado</code> en el panel.
2	Editar los datos generales del producto	Nombre, descripción, categoría, proveedor e imagen desde el panel.
3	Alinear la base de datos con el EER	Migración nueva que quite el trigger de especialización disjunta (el EER la marca traslapada) y pase la llave del historial a <code>estado</code> , con sus pruebas y semillas; corregir también la página «Mapeo» del diagrama (§ 3.3).
4	Decisiones prioritarias del modelo	Referencia de un pago sin respuesta de la pasarela, usuarios que no son ni administrador ni cliente, permisos de una cuenta que es administrador y cliente, y dónde vive la bitácora.
5	Ingreso de mercancía y alerta (HU-07, HU-08)	Con ellos cada entrada de mercancía queda en <code>producto_administra</code> (hoy el panel corrige las existencias sin dejar ese registro), los productos importados tendrán existencias sin pasar por <code>db:cliente</code> , y «Recién llegados» podrá ordenarse por fecha.
6	Total y paginación del catálogo público	Agregar el total a la respuesta de <code>GET /api/catalogo/productos</code> y decidir entre números de página o «Cargar más».
7	Recuperación de contraseña y cuentas de empleados	RF-54, cierre de sesión con la política de contraseñas del cliente y el rol empleado.
8	Limpiezas	Semillas en dólares, comentarios que todavía dicen «colones», pasar <code>api/sesion.js</code> y <code>api/importaciones.js</code> al cliente HTTP, logo y favicon definitivos.

Este documento se hizo con ayuda de Claude (Anthropic) para ordenar la información del repositorio y de Jira, correr las pruebas y tomar las capturas de la evidencia. Las decisiones, el código y el contenido de la retrospectiva son del equipo.